



Fuzzing RTOS and Embedded IPC: Vulnerability Research for Less Documented Protocols and Systems

Kashif Amanat and Naveenkumar Angappan

BSides Krakow

Workshop Roadmap & Interactive Sessions

- Basic Concepts: Fuzzing Overview - Web (relatable for everyone), hardware
- Challenges in Fuzzing: Issues like undocumented or less documented protocols/low level
- Attack Surface in RTOS
- Journey of Exploiting a Test System
- Take Aways :)



Note about Labs: While labs are designed to be interactive, I will also provide live demonstrations for those who would prefer to observe.

Recommendation to Start Practical Setup Downloads and Setup

- Download [set-upV2.sh](https://github.com/kashif-23/fuzzingRTOS_Workshop) Script from: https://github.com/kashif-23/fuzzingRTOS_Workshop
- `chmod +x set-upV2.sh`
- `./set-upV2.sh`



Note: This script sets up west and installs required RTOS lab set up required for practical part of the workshop

Note about Labs: While labs are designed to be interactive, I will also provide live demonstrations for those who would prefer to observe.



My Introduction



Naveenkumar Angappan

Offensive Security Engineer



- 9+ years in offensive security across red teaming and embedded systems, with multiple pentests.
- CISSP, OSCP , currently pursuing OSAI with focus on AI red teaming

- https://github.com/kashif-23/fuzzingRTOS_Workshop

Workshop Engagement

I. Communications

Live Q&A: Post and upvote questions on this QR

Hygiene: If the presentation answers your question,
or it becomes irrelevant, please delete it from the queue.



II. Operational Modes

Choose your level of involvement based on your interest:

Mode A: Recon (Observation)

Watch the live execution and telemetry on the main screen.

Focus on Fuzzing Theory.

Mode B: Operator (Active Follow)

Execute labs in real-time.

Workshop Engagement



- https://github.com/kashif-23/fuzzingRTOS_Workshop

Rules and Disclaimer

The Rules

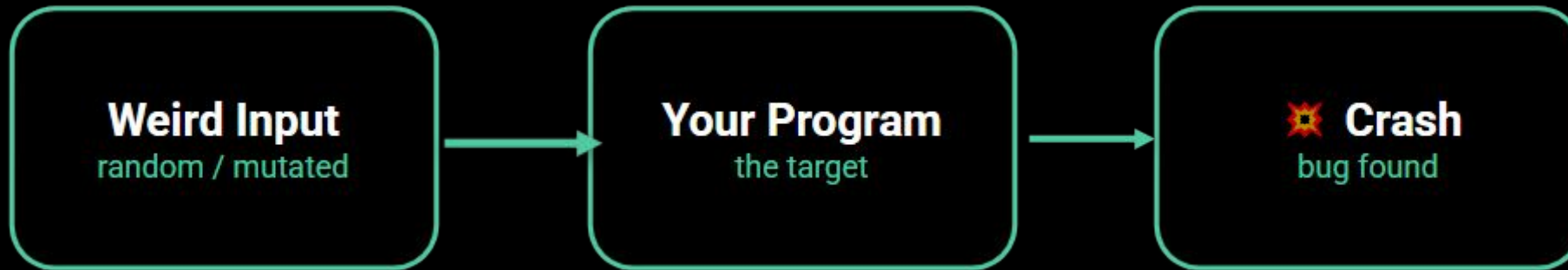
- Containment: Keep all code within Lab VMs or personal hardware.
- Operational Security: Do not upload binaries to VirusTotal/Public Sandboxes.

Disclaimer

- Educational Use Only: For research and authorized testing against hardened targets.
- No Liability: Presenters and BSides are not responsible for any misuse.
- Ethics: Participants must adhere to professional and legal standards

Fuzz Testing

- What it is: feed a program input it wasn't built for — random, mutated, or rule-based — and see if it breaks.
- Where it started: in 1990, a basic random-input tool crashed about 1 in 3 standard Unix programs.
- Why it works: real bugs hide in edge cases nobody tested. A fuzzer can try millions of them while you sleep.



Automated — repeated millions of times, faster than any human could test by hand.

Why do we need to Fuzz

Fewer False Alarms

Static analysis and code review flag a lot of noise. Fuzzing proves a bug is real by actually breaking the program.

Cheap Once Built

Write the harness once, and it keeps testing for free, continuously, in CI.

Explores on Its Own

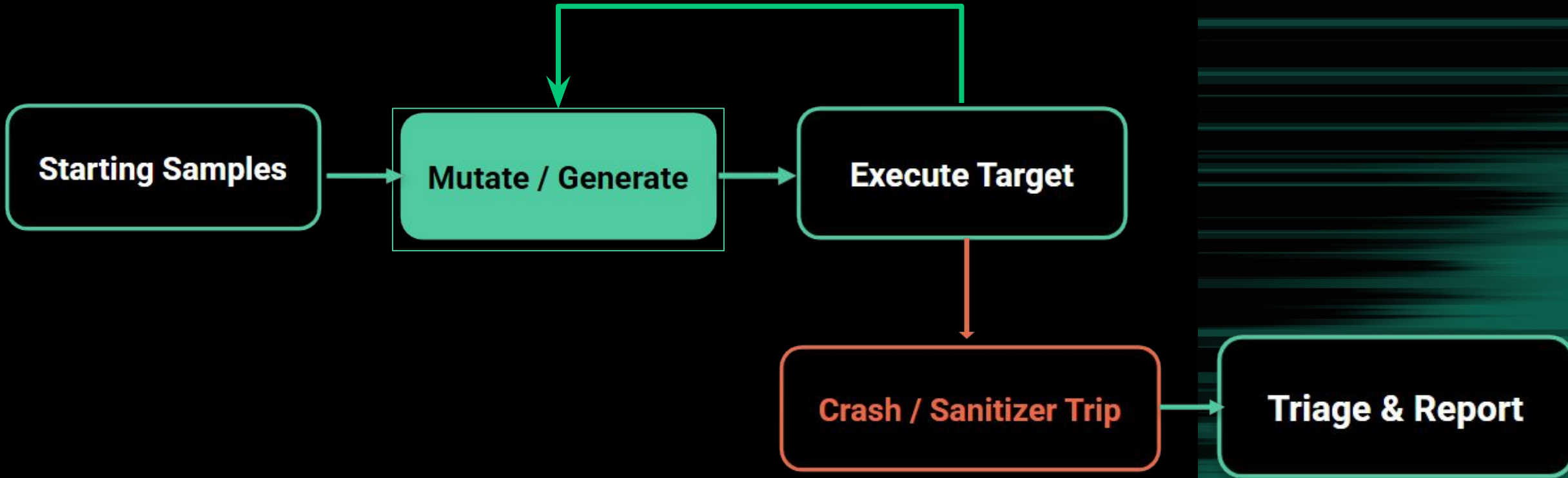
Rewards any input that reaches new code — finding cases you'd never think to write a test for.

Still the Top CVE Source

C and C++ still run embedded, kernel and RTOS code, where memory bugs — overflows, OOB access, use-after-free — are still the majority of CVEs.

How to Fuzz

new coverage feeds the samples



Interesting inputs (new coverage) feed back into the corpus – the loop gets smarter about your code the longer it runs.

Types of Fuzzer

Categories	Option A	Option B
Feedback	Black-box — no visibility into execution, inputs generated blind.	Grey-box — coverage-guided via lightweight instrumentation (the AFL / libFuzzer model).
Input source	Mutation-based — start from real seeds, flip/splice bytes.	Generation-based — build inputs from a grammar/protocol spec.
Analysis depth	Random / dumb — cheap, wide, shallow.	Symbolic / concolic — solves constraints to reach deep paths, expensive per input.

Common Fuzzing Techniques & Tools



Coverage-Guided

AFL++

honggfuzz

libFuzzer

Watches which code runs and steers mutations toward new paths.
Powers today's demo.



Structure-Aware

Peach Fuzzer

boofuzz

RESTler

You describe the message format; the fuzzer mutates within those rules.

Kernel & Syscall

syzkaller

Generates syscall sequences — the closest thing to fuzzing an RTOS's own API surface.



The Oracle: Sanitizers

ASan

UBSan

MSan

Turn silent memory corruption into a loud, instant crash.

Fuzzing is (relatively) easy — for web apps & networks

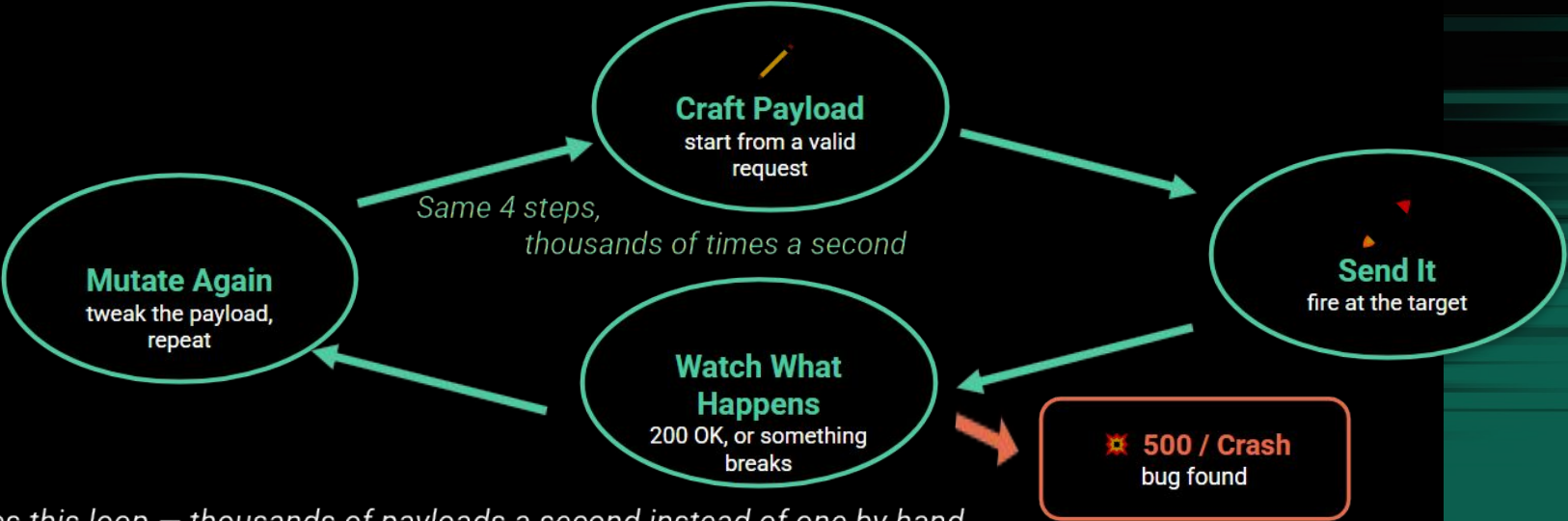
**Mature Tools Exist**
Burp Suite, ZAP, ffuf, sqlmap.

Protocols Are Text
HTTP, JSON, GraphQL — easy to parse.

Cheap Targets
Thousands of containers in parallel.

**Crashes Are Obvious**
Exit code, 500, or core dump.

**Free Samples**
Captured traffic = free seed data.



A fuzzer just automates this loop — thousands of payloads a second instead of one by hand.

Why Fuzz RTOS Specifically?

Runs Critical Systems

Cars (AUTOSAR ECUs), factory PLCs, medical devices, and avionics all run on top of an RTOS.

IPC Is the Trust Boundary

Queues and message-passing are exactly where data crosses from one task into another.

No Walls Between Tasks

No MMU, one shared address space — one task's IPC bug can corrupt the kernel, or the whole system.

Already Happening

RT-Fuzzer (2026) found a CVSS 8.7 bug in a real Schneider Electric PLC, plus two bugs in Apache NuttX's filesystem.

Challenges in Fuzzing RTOS

Unpredictable Timing

Interrupts jump in, so the same input can take a different path each run.

Weird Custom Formats

CAN, SOME-IP, proprietary buses — every channel has its own framing.

Blind Without Help

Real hardware won't show you code coverage on its own.

No Crash Signal

No SIGSEGV, no core dump — the damage shows up somewhere else entirely.

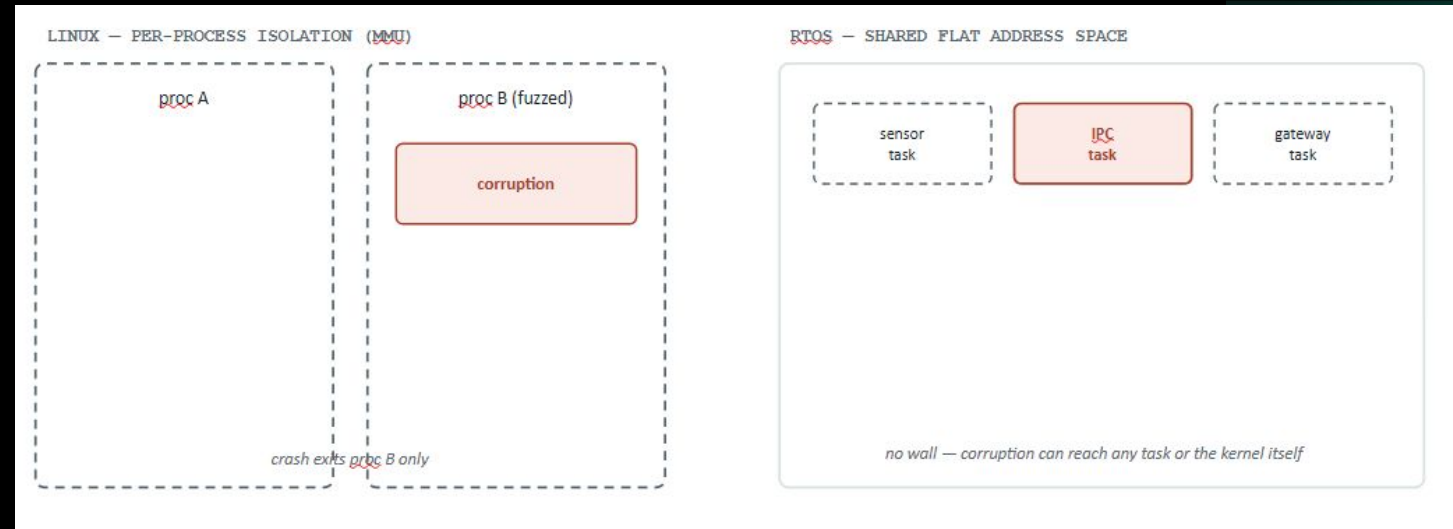
One Shared Crash Space

Fuzzer and target share memory — a bad input can wedge the whole board.

Five things that make an RTOS a harder target than a web app.

One Bug, Whose Memory?

- Linux — per-process isolation (MMU): the fuzzed process is walled off from every other process; a crash exits that process alone.
- RTOS — shared flat address space: sensor task, IPC task and gateway task all live in the same memory with no wall between them.
- No wall means no containment: corruption inside one IPC call can reach any other task, or the kernel itself — not just the code being fuzzed.



IPC Across RTOSes

RTOS	Core primitives	Typical execution model	Main IPC model	Memory Isolation	Fuzz-relevant detail
QNX	MsgSend / MsgReceive / MsgReply, pulses, POSIX mqueues, shared memory	Processes + threads	Process-to-process message passing	Strong	fuzz messages crossing between isolated processes
FreeRTOS	xQueueSend/Receive, semaphores & mutexes (built on queues), task notifications, stream & message buffers	Tasks in one application/address space	Task-to-task queues/notifications	Usually weak/flat	fuzz data moving through queues and buffers between tasks.
Zephyr	k_msgq, k_pipe, k_mbox, k_fifo/k_lifo, IPC service (OpenAMP)	Mostly threads, optionally isolated memory domains	Thread-to-thread kernel objects	Medium Configurable	fuzz buffer-and-length based interfaces and check how threads handle malformed data.

Practical Demonstration and Lab Work



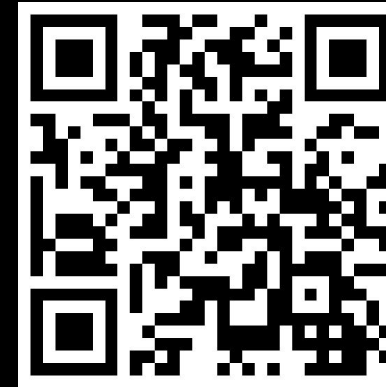
Github

My Introduction



Kashif Amanat

Offensive Security Engineer

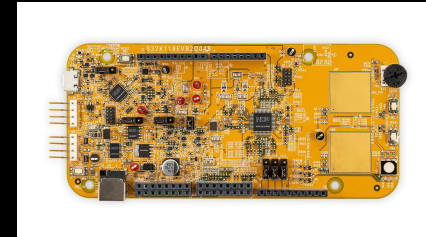


/in/kashifamanat

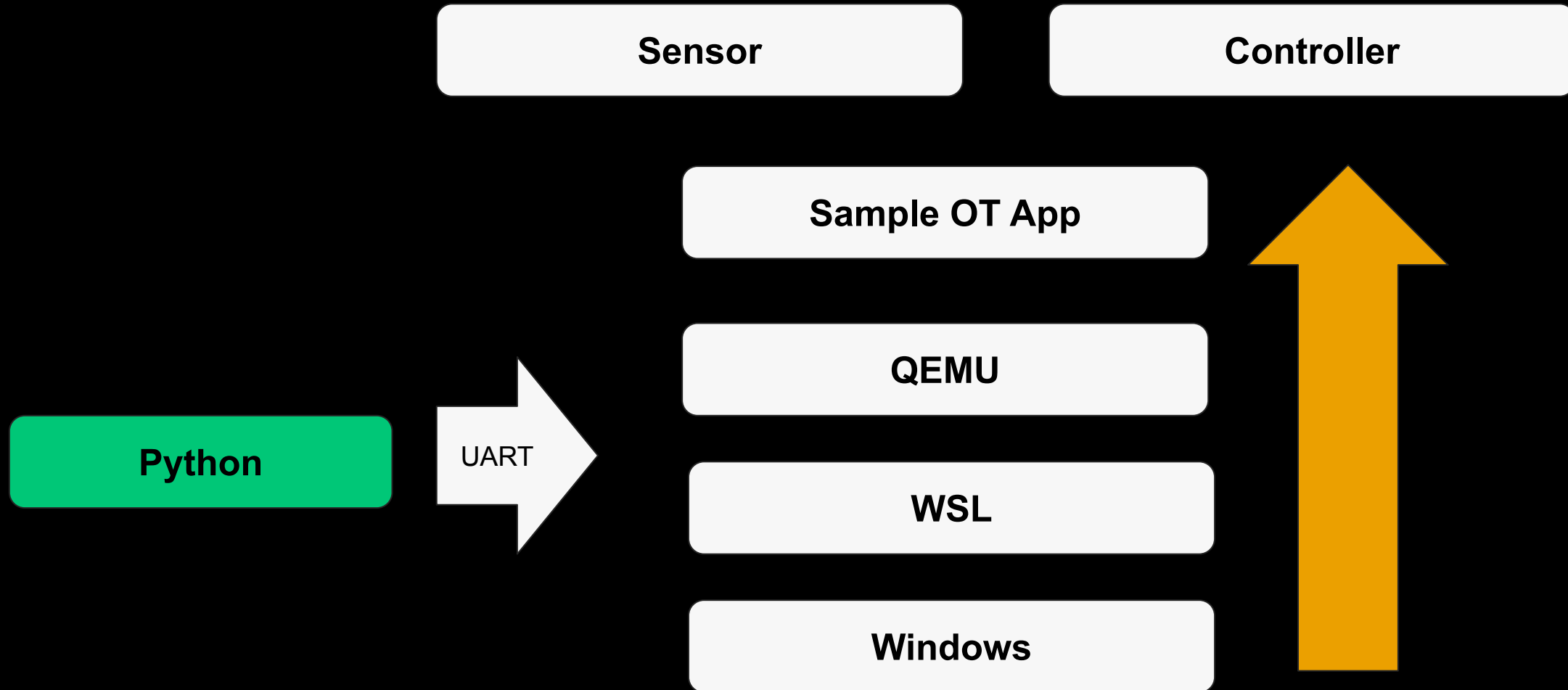
- 10+ years in offensive security across red teaming and embedded systems, with multiple pentests, bug bounties and real-world CVEs
- Specialized in execution flow and evasion techniques: secure boot bypass, firmware reversing, and environment-aware behavior analysis
- OffSec certified (OSEP, OSCP), currently pursuing OSED with focus on exploit development and stealth execution logic

Scenario

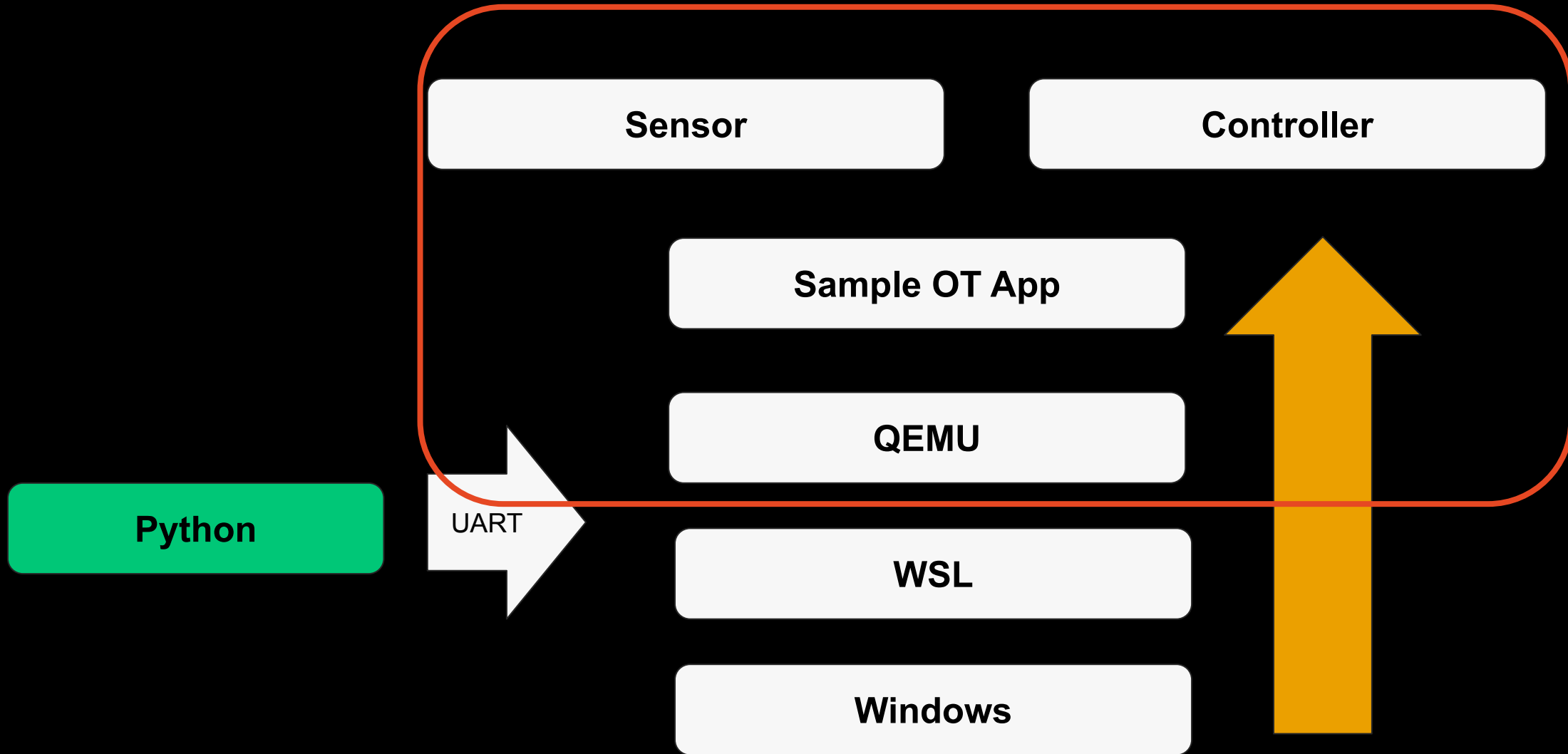
- We have an embedded device (IoT, Medical, OT, Automotive, Cellular, GNSS)
- We need to find a way to exploit it / like crash, control or manipulate its software.
- First step would be to access an interface like UART. For today's learning, we have a virtualized demonstration for workshop.



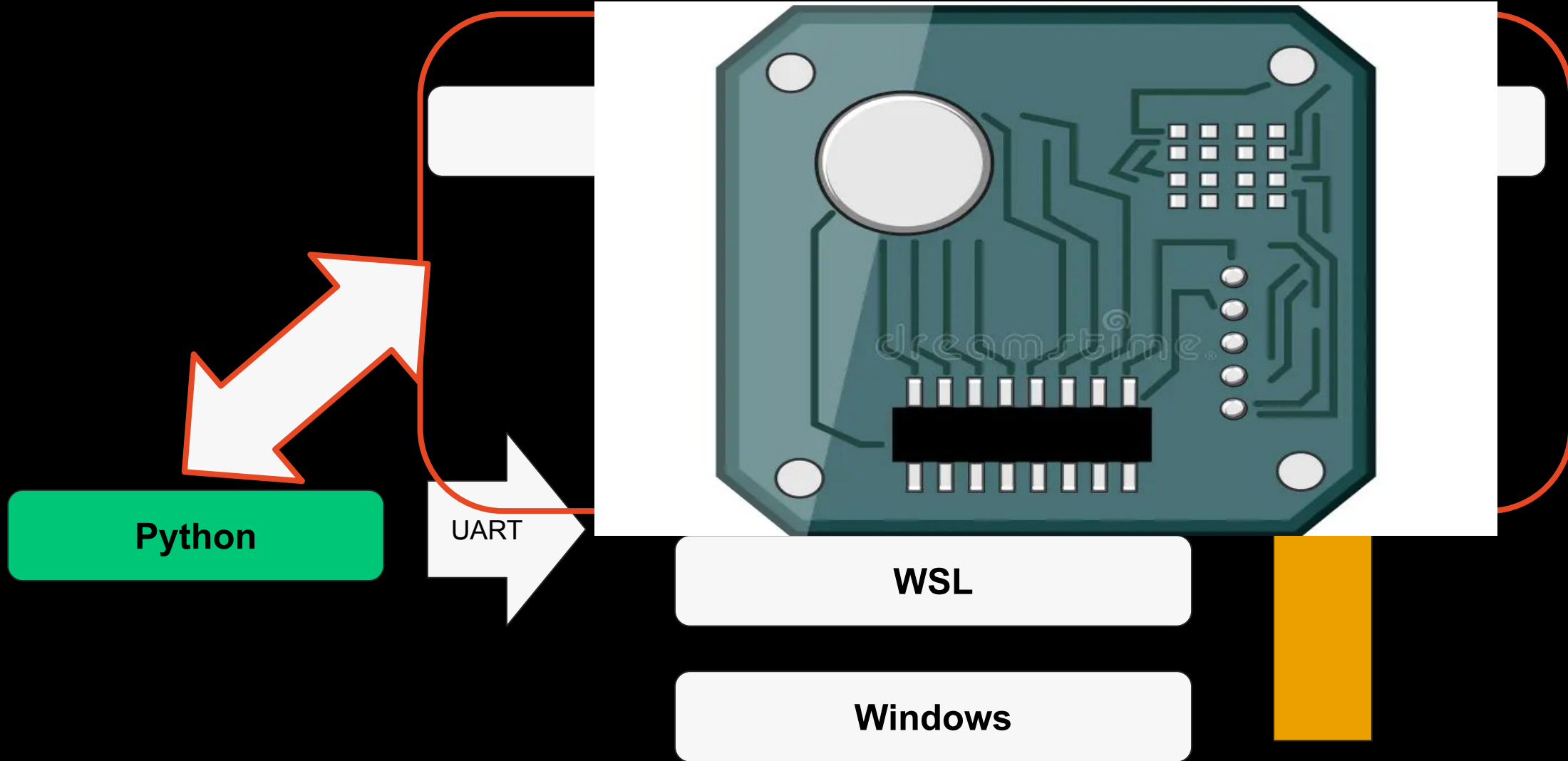
Virtualized Environment



Virtualized Environment



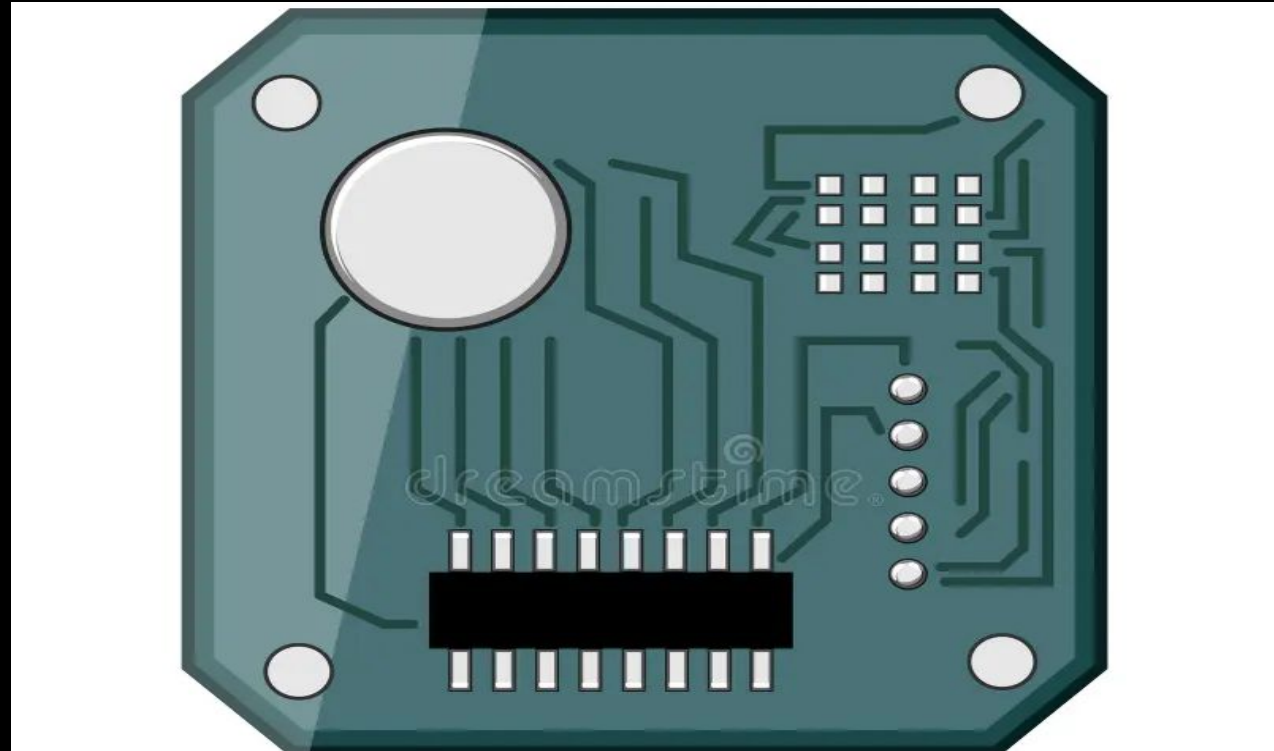
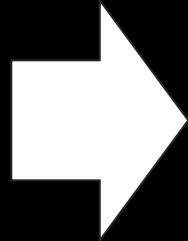
Virtualized Environment



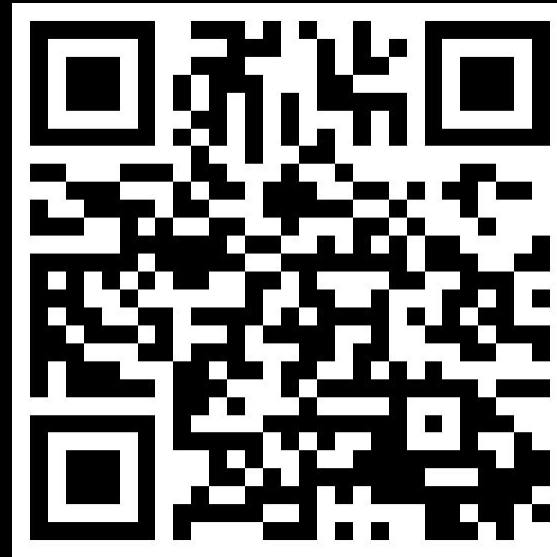
If you Followed, We are Doing This:



Attacker



Live Demonstration



Zephyr RTOS Attack Surface

- Message Queues (k_msgq_put()/k_msgq_get())
Fixed-size messages.
Risk: wrong message size can cause memory handling issues.
- Pipes (k_pipe_put() / k_pipe_get())
Transfer variable-length byte streams.
Risk: malformed length or buffer data can trigger overflows or crashes.
- Mailboxes (k_mbox_put() / k_mbox_get())
Pass messages between threads.
Risk: sender and receiver may interpret the message size differently.
- ISR IPC Paths
Some IPC APIs can also be called from interrupts.
Risk: this creates an additional code path that should be tested separately.
 - Most IPC interfaces handle:
Buffer + Length
 - A fuzzer can easily mutate both:
Data + Size → IPC → Receiving Thread → Parser

01 / FROM UART TO AN ATTACK SURFACE

The assessment starts with an interface — not with source code.



WHAT WE CAN OBSERVE

```
$ help
```

```
controller ipc sensor diag  
device kernel shell
```

MENTAL MODEL



Security mindset

We do not need to understand everything.

We look for externally reachable behavior that crosses trust boundaries.

NEXT QUESTION: What happens if we send something the developer did not expect?

02 / WHAT IS ACTUALLY HAPPENING INSIDE?

Once the shell exposes an IPC command, we can trace the message through the RTOS.

APPLICATION

SENSOR THREAD

Produces a structured message

command = 0x10
length = 8
sensor_id = 1001

k_msgq_put()
enqueue

ZEPHYR k_msgq

depth = 16
fixed-size items

k_msgq_get()
dequeue

CONTROLLER THREAD

Receives message
calls parser

APPLICATION PROTOCOL

magic | command | length | sensor_id | payload[32]

IMPORTANT DISTINCTION k_msgq is real Zephyr IPC. The application message format is our lab protocol.

03 / FROM MALFORMED INPUT TO CRASH

The black-box observation becomes explainable once we trace the data path.



ROOT-CAUSE ZOOM



ASSESSMENT PATTERN observe → mutate → reproduce → inspect source → explain

WHY 17?

16 bytes fit. The next byte crosses the destination boundary.

04 / FROM ONE CRASH TO SYSTEMATIC FUZZING

We do not start with random bytes — we start with an understanding of the protocol.

01

MODEL

Understand fields

02

MUTATE

8 → 16 → 17 → ...

03

AUTOMATE

Generate valid messages

04

OBSERVE

Accept / reject / crash

05

MINIMIZE

Find smallest trigger

WHY PROTOCOL-AWARE FUZZING?

Random bytes often die before reaching interesting code.

Structured mutations preserve enough protocol validity to exercise the parser.

Our target: mutate `length` while keeping the rest of the message valid.

SAME QUESTIONS, DIFFERENT RTOS

ZEPHYR

Thread → `k_msgq` → Thread

`k_msgq_put()` → `k_msgq_get()`

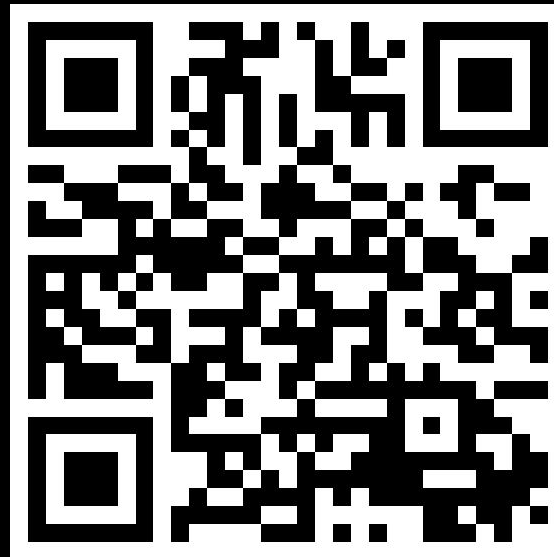
QNX

Process → message passing → Process

`MsgSend()` → `MsgReceive()` → `MsgReply()`

SECURITY QUESTIONS What crosses the boundary? Who can send it? What does the receiver trust? Are lengths validated?

Analysis to Find a Way to Speak with the Device



Github

Key Takeaways

- Fuzzing finds the inputs you didn't think to write — true whether the target is a REST endpoint or a message queue.
- IPC is the highest-value target in an RTOS — it's where task boundaries, and therefore trust boundaries, actually sit.
- RTOS fuzzing breaks the usual assumptions — no MMU, no standard crash signal, non-deterministic scheduling — isolate a task into a harness instead.
- Host-simulated targets are the fastest way in — Zephyr's `native_sim` gives you libFuzzer, AFL++ and full sanitizer coverage before touching real hardware.
- The API shape gives it away — any function taking a buffer and a length is a fuzzing target waiting for a harness.

References

- Zephyr Project — Exploring Fuzzing for Zephyr Applications with libFuzzer and AFL++.
- Zephyr Project Documentation — Fuzzing sample (samples/subsys/debug/fuzz).
- NDSS 2026 — RT-Fuzzer: Task-Driven Fuzzing of RTOS Firmware.
- Muench et al. — What You Corrupt Is Not What You Crash: Challenges in Fuzzing Embedded Devices.
- QNX docs — MsgSend / MsgReceive / MsgReply. FreeRTOS docs — queues, stream & message buffers.



Feedback

Q & A and Brainstorming